

المرحلة 8

المرحلة 8: التقدّم والتنزيلات والخزانة

تقدّمي · الشارات · الخزانة وشراء الزينة بالنجوم · شاشة إدارة التنزيلات

الشاشات: 17 تقدّمي · 18 التنزيلات · 24 الخزانة

يسبقها: المرحلة 7 (ب)

المشروع: رِزمة · تطبيق تعلّم يعمل دون إنترنت

وقت القراءة: 50 دقيقة

تتبعها: المرحلة 9

المنصة: Flutter · Dart · Drift · Riverpod · go_router

1. ما هذه المرحلة؟

المرحلة 8 تعطي الطالب **إحساس التقدّم والمكافأة**: شاشة **«تقدّمي»** التي تلخّص ما أنجزه، و**«الخزانة»** التي يصرف فيها نجومه على إكسسوارات وألوان لشخصية رِزمة، وشاشة **«إدارة التنزيلات»** التي تُظهر ما على جهازه وتتحكم به.

محاولات وأجزاء منجزة ◄ ProgressState.derive ◄ تقدّمي (17) ◄ النجوم ◄ الخزانة (24)

الشاشات التي بُنيت

الشاشة	المسار	الملف	ما تفعله
17 تقدّمي (تبويب)	`/progress`	`progress/progress_screen.dart`	ملخص التقدّم: العام، النجوم، النشاط الأسبوعي، المواد، المراجعة، الشارات
18 إدارة التنزيلات	`/account/downloads`	`account/downloads_screen.dart`	ما على الجهاز، المساحة، التحديثات، المزامنة، المشاركة
اختيار ما يُنزَّل	`/account/downloads/choose`	`account/choose_downloads_screen.dart`	اختيار المواد والدروس للتنزيل
24 خزانة رِزمة	`/account/wardrobe`	`wardrobe/wardrobe_screen.dart`	شراء ولبس إكسسوارات وألوان بالنجوم

> **ملاحظة**
>
> نظام التنزيل نفسه (الخدمات، التوقيع، التخزين) يشرحه **دليل تنزيل الدروس**. هنا نغطي الشاشة وما تعرضه.

الملفات الأساسية

الملف	دوره
`progress/progress_state.dart`	دالة صرفة `ProgressState.derive` تحسب كل شيء
`progress/progress_providers.dart`	`progressStateProvider` يجمع المصادر
`progress/progress_screen.dart`	الشاشة (601 سطرًا بأقسام منفصلة)
`wardrobe/wardrobe_items.dart`	تعريف العناصر وأسعارها: `wardrobeItems`
`wardrobe/wardrobe_repository.dart`	الرصيد، الحالة، الشراء (معاملة)، اللبس

الملف	دوره
`wardrobe/wardrobe_screen.dart` / `wardrobe_providers.dart`	الواجهة والمزوّدات
`core/widgets/mascot_items.dart`	رسم الإكسسوارات على التميمة بالكود
`core/widgets/progress_ring.dart`	حلقة نسبة التقدّم
`core/db/tables/owned_items.dart` / `daos/wardrobe_dao.dart`	المشتريات

2. تقدّمي (17)



تقدّمي



التنزيلات



الخزانة

مثل الرئيسية: **كل شيء مشتق**. دالة صرفة `ProgressState.derive` تأخذ الملف والكتالوج والتقدّم والمحاولات والأجزاء والإتقان والوقت وتعيد كائنًا بكل ما تعرضه الشاشة:

lib/features/progress/progress_state.dart سطر 28

```
class ProgressState {
  const ProgressState({
    required this.grade,
    required this.overall,
    required this.hasActivity,
    required this.streak,
    required this.completedLessons,
    required this.reviewCount,
    required this.quizzes,
    required this.stars,
    required this.week,
    required this.subjects,
    required this.review,
    required this.badges,
  });

  final int grade;

  final double overall;
  final bool hasActivity;
  final int streak;
  final int completedLessons;
  final int reviewCount;
  final int quizzes;
  final int stars;

  final List<int> week;
  final List<SubjectBar> subjects;
  final List<ReviewItem> review;
  final List<BadgeStatus> badges;
```

القسم	كيف يُحسب
التقدّم العام	متوسط تقدّم الدروس **التي لها محتوى** فقط (وإلا يبقى صفرًا بلا سبب)
بطاقة النجوم	مجموع starsEarned لكل المحاولات
بلاطات الإحصاء	دروس مكتملة · مفاهيم للمراجعة (<60%) · عدد الاختبارات
نشاط الأسبوع	عدد الأنشطة (محاولات + أجزاء منجزة) لكل يوم، والأسبوع يبدأ **بالسبت**
حسب المادة	متوسط تقدّم دروس المادة، و«قريبًا» لمادة بلا محتوى
يحتاج مراجعة	أضعف 5 مفاهيم؛ النقر يفتح خطة المراجعة
الشارات	تُشتق من البيانات ولا تُخزَّن (الجدول أدناه)

النشاط الأسبوعي

lib/features/progress/progress_state.dart سطر 61

```
static int weekdayIndex(DateTime d) =>
    (d.weekday - DateTime.saturday + 7) % 7;
```

نحسب بداية الأسبوع (السبت) ثم نضع كل نشاط في خانة اليوم: `i = day.difference(weekStart).inDays`. والنتيجة قائمة من 7 أعداد ترسمها الشاشة أعمدة.

> **لماذا؟**
>
> المخطط يعدّ «أنشطة» لا «دقائق» لأننا
>
> **لا نقيس وقتًا**
>
> . نصرّح بما نعرفه فقط (صدق في العرض).

الشارات الخمس

الشارة	الشرط
درسك الأول	دروس مكتملة > 0
3 أيام متتالية	السلسلة ≥ 3
100 نجمة	مجموع النجوم ≥ 100
مستمع ماهر	أنجز جزء «الكلمات» في أي درس
اختبار كامل	اختبار بالدرجة الكاملة (score == total)

lib/features/progress/progress_state.dart سطر 159

```
  badges: [
    BadgeStatus(BadgeKind.firstLesson, completed > 0),
    BadgeStatus(BadgeKind.streak, streak >= 3),
    BadgeStatus(BadgeKind.stars, stars >= 100),
    BadgeStatus(
      BadgeKind.listener,
      parts.any((p) => p.part == LessonPart.words),
    ),
    BadgeStatus(
      BadgeKind.quizAce,
      quizAttempts.any((a) => a.total > 0 && a.score == a.total),
    ),
  ],
);
```

> **نصيحة**
>
> لإضافة شارة جديدة: (1) أضف قيمة إلى `BadgeKind` (2) أضف سطرًا في قائمة `badges` بشرطها (3) نصّها في `ArProgress` (4) اختبارها في `progress_test.dart`. لا جدول ولا ترحيل.

المزوّد

lib/features/progress/progress_providers.dart سطر 11

```
final progressStateProvider = Provider.autoDispose<ProgressState?>((ref) {
  final profile = ref.watch(activeProfileProvider).value;
  final catalog = ref.watch(catalogProvider).value;
  final content = ref.watch(contentIndexProvider).value;
  if (profile == null || catalog == null || content == null) return null;

  final progress = ref.watch(lessonProgressProvider(profile.id)).value;
  final parts = ref.watch(lessonPartsProvider(profile.id)).value;
  final attempts = ref.watch(attemptsProvider(profile.id)).value;
  final mastery = ref.watch(masteryProvider(profile.id)).value;
  if (progress == null ||
      parts == null ||
      attempts == null ||
      mastery == null) {
    return null;
  }

  return ProgressState.derive(
    profile: profile,
    catalog: catalog,
```

مثل `homeStateProvider` : يجمع Streams ويعيد `AsyncValue` ، فتتحدّث الشاشة مع أي محاولة جديدة.

3. الخزانة (24)

النجوم التي يكسبها الطالب من التدريب والاختبار **تُصرف** على زينة لشخصيته. وهي تفتح الزينة **فقط**؛ الدروس مفتوحة دائمًا (مكتوب في الشاشة).

العناصر وأسعارها

العنصر	النوع	السعر	ملاحظة
قبعة التخرّج	إكسسوار	مجانية	
الشال المطرّز	إكسسوار	20	
نظارة القراءة	إكسسوار	30	
نجمة الإتقان	إكسسوار	مجانية	تُفتح بإكمال درس
أخضر	لون	مجاني	اللون الافتراضي
برتقالي · سماوي	لون	40	
بنفسجي · عسلي	لون	50	

lib/features/wardrobe/wardrobe_items.dart سطر 8

```dart
class WardrobeItem {
  const WardrobeItem({
    required this.id,
    required this.kind,
    required this.name,
    required this.price,
    this.unlock = Unlock.none,
    this.color,
  });

  final String id;
  final WardrobeKind kind;
  final String name;

  final int price;
  final Unlock unlock;

  final ProfileColor? color;
```

الرصيد = النجوم − ما أُنفق

lib/features/wardrobe/wardrobe_repository.dart سطر 33

```dart
static int balance({
  required int stars,
  required Iterable<OwnedItem> owned,
}) => rawBalance(stars: stars, owned: owned).clamp(0, 1 << 31);
```

- الرصيد **يُحسب ولا يُخزَّن**: مجموع نجوم المحاولات ناقص مجموع `OwnedItem.cost`.
- تكلفة العنصر **تُحفظ وقت الشراء** (`cost` في `owned_items`) فلا يتغيّر ما دفعه الطالب لو تغيّر السعر لاحقًا.
- ويُقيَّد الرصيد بـ `clamp(0, ...)` كي لا يظهر سالبًا.

حالة العنصر

lib/features/wardrobe/wardrobe_repository.dart سطر 38

```
static ItemState stateOf(
  WardrobeItem item, {
  required Set<String> owned,
  required Set<String> equipped,
  required bool lessonCompleted,
  required String currentColorItemId,
}) {
  final isOwned =
      owned.contains(item.id) ||
      (item.unlock == Unlock.lessonCompleted &&
          lessonCompleted &&
          item.price == 0);
  if (isOwned) {
    final on = item.kind == WardrobeKind.color
        ? item.id == currentColorItemId
        : equipped.contains(item.id);
    return on ? ItemState.equipped : ItemState.owned;
  }
  if (item.unlock == Unlock.lessonCompleted && !lessonCompleted) {
    return ItemState.locked;
  }
  return ItemState.buyable;
```

الحالة	المعنى
`equipped`	مملوك ومُلبّس الآن
`owned`	مملوك وغير مُلبّس
`buyable`	متاح للشراء
`locked`	يحتاج فتحًا (مثل نجمة الإتقان: إكمال درس)

الشراء معاملة واحدة

lib/features/wardrobe/wardrobe_repository.dart سطر 80

```
Future<PurchaseResult> buy(String profileId, String itemId) {
  final item = wardrobeItemById(itemId);
  if (item == null) return Future.value(PurchaseResult.unknownItem);
  return _db.transaction(() async {
    final owned = await _db.wardrobeDao.getOwned(profileId);
    if (ownedIds(owned).contains(itemId)) return PurchaseResult.alreadyOwned;
    if (item.unlock == Unlock.lessonCompleted &&
        !await _lessonCompleted(profileId)) {
      return PurchaseResult.locked;
    }
    final stars = await _db.activityDao.totalStars(profileId);
    if (rawBalance(stars: stars, owned: owned) < item.price) {
      return PurchaseResult.notEnoughStars;
    }
    final ok = await _db.wardrobeDao.insertPurchase(
      profileId: profileId,
      itemId: itemId,
      cost: item.price,
      now: _now(),
    );
    return ok ? PurchaseResult.purchased : PurchaseResult.alreadyOwned;
  });
}

Future<bool> toggleAccessory(String profileId, String itemId) async {
```

1. عنصر مجهول ← `unknownItem` .
2. مملوك أصلًا ← `alreadyOwned` .
3. مقفل ولم يُفتح ← `locked` .
4. الرصيد أقل من السعر ← `notEnoughStars` .
5. وإلا نكتب الشراء. والمفتاح المركّب **(الطالب، العنصر)** في الجدول يمنع الشراء المزدوج حتى لو نُقر الزر مرتين بسرعة (اختبار بنقرتين متزامنتين).

> **لماذا؟**
>
> الفحوص والكتابة داخل
> **معاملة واحدة**
> فلا يتغيّر الرصيد بين الفحص والخصم. وكون النتيجة قيمة من `PurchaseResult` (لا استثناء) يجعل الواجهة تعرض رسالة مناسبة لكل حالة.

اللبس واللون والرسم

- **اللبس:** يُحفظ في عمود `equipped` في ملف الطالب (`ProfilesDao.decodeEquipped`)، فيظهر ما يرتديه في كل مكان تظهر فيه التميمة (الرئيسية، حسابي، اختيار الملف).
- **اللون:** `useColor` يغيّر لون الملف كله (ProfileColor)، وليس زينة فقط.

- **الرسم:** الإكسسوارات مرسومة **بالكود** (`mascot_items.dart`) على لوحة 150×150 نفسها التي رُسمت عليها التميمة، فتتطابق مواضعها عند أي حجم.

4. إدارة التنزيلات (18)

كانت هذه الشاشة في بدايتها «حالة فارغة» لأن المحتوى جزء من التطبيق. الآن صارت **مركز التحكم بالمحتوى المنزَّل**. تعرض أقسامًا:

القسم	ما يعرضه
`_Banner`	لافتة حالة: «كل المحتوى على جهازك» أو ما ينقص
`_Card` (المساحة)	حجم ما على الجهاز ونسبة المحمَّل من الكتالوج
`_InstalledRow`	صف لكل درس منزَّل بأجزائه وأحجامه
`_SyncCard`	حالة النتائج بانتظار الاتصال والمزامنة
`_ShareCard`	إرسال المحتوى لهاتف آخر أو استلامه (ملف .rizma أو Wi-Fi)
`_TipCard`	نصائح (تعلّم دون انقطاع)

وزر «اختيار ما يُنزَّل» يفتح `ChooseDownloadsScreen` لانتقاء المواد والدروس. وتفضيلات «Wi-Fi فقط» و«تنزيل الفيديو تلقائيًا» و«التحديث التلقائي» تأتي من `DownloadPrefs` (المرحلة 4).

> **انتبه**
>
> كل ما يخص
> **كيف**
> يُنزَّل المحتوى (الخادم، توقيع الفهرس، البصمات، الاستئناف) وكيف يُشارَك بين الهواتف موجود في دليلي
> **تنزيل الدروس**
> و
> **المشاركة بين الهواتف**
> . لا تعدّل شيئًا في الخدمات من هذه الشاشة.

5. الاختبارات وطريقة التجربة

الملف	ما يغطيه
`test/features/progress_test.dart` (7)	اشتقاق «تقدّمي»: التقدّم العام، الأسبوع، الشارات، المراجعة، مادة بلا محتوى
`test/features/wardrobe_test.dart` (10)	الشراء والرصيد واللبس واللون والقفل وحذف الملف، ومنع الشراء المزدوج
`test/features/downloads_ui_test.dart`	شاشة التنزيلات بحالاتها
`test/account_test.dart`	فتح التنزيلات من حسابي

الملف	ما يغطيه
`test/layout_stress_learning_test.dart`	تقدّمي والتنزيلات والخزانة على 320×568 بخطين

> **نصيحة**
> شغّل: `flutter test test/features/progress_test.dart test/features/wardrobe_test.dart`

جرّبها بنفسك

1. طالب جديد ← «تقدّمي»: كل شيء صفر والمواد «لم تبدأ».
2. أكمل جولة تدريب ← النجوم تزيد؛ ونشاط اليوم يرتفع عمودٌ في الأسبوع.
3. افتح «الخزانة» ← الرصيد = نجومك؛ اشترِ «الشال» (20) إن كفى ← يُلبَس على التميمة في «حسابي».
4. غيّر اللون إلى برتقالي ← يتبدّل لون الشخصية في كل الشاشات.
5. افتح «إدارة التنزيلات» من «حسابي» ← الحجم والدروس المنزَّلة.

6. تدرّب وتحقّق من فهمك

قرارات تصميم وأسبابها

القرار	السبب
ProgressState دالة صرفة	تُختبر بلا واجهة وبتواريخ ثابتة.
الشارات تُشتق ولا تُخزَّن	تُعاد من البيانات فلا تحتاج مزامنة ولا ترحيلًا.
الأسبوع يبدأ بالسبت	يوافق أسبوع المدرسة عندنا.
الرصيد يُحسب ولا يُخزَّن	لا يختلف عن مصدره (المحاولات والمشتريات).
تكلفة العنصر تُحفظ وقت الشراء	تغيير السعر لاحقًا لا يغيّر ما دفعه الطالب.
الإكسسوارات بالكود لا صور	تتطابق المواضع عند أي حجم ولا حاجة لملف لكل حالة.

أسئلة

س1. نال طالب 100 نجمة وأنفق 70. كم رصيده؟ وهل يستطيع شراء لون بـ 40؟

س2. ما الذي يمنع الخصم المزدوج عند نقر زر «شراء» مرتين بسرعة؟

س3. لماذا يُقاس «التقدّم العام» على الدروس التي لها محتوى فقط؟

س4. متى تُفتح نجمة الإتقان؟ وماذا تكون حالتها قبل ذلك؟

س5. لماذا يُعدّ «نشاط الأسبوع» أنشطةً لا دقائق؟

تمارين

1. أضف عنصر خزانة جديدًا (مثل «سماعات» بـ 25) في `wardrobeItems` وارسمه في `mascot_items.dart`.
2. اكتب اختبارًا في `wardrobe_test.dart` أن الشراء بنجوم أقل من السعر يعيد `notEnoughStars` ولا يخصم شيئًا.
3. أضف شارة «5 اختبارات» بشرط `quizAttempts.length >= 5` واختبرها.

الإجابات

ج1. الرصيد 30 ((70 – 100، فلا يستطيع شراء لون بـ 40.

ج2. المفتاح الأساسي المركّب (الطالب، العنصر) في جدول owned_items مع معاملة الشراء: الثانية تجد العنصر مملوكًا فتعيد alreadyOwned.

ج3. لأن الدروس بلا محتوى لا يمكن إكمالها، فإدخالها في المتوسط يُبقيه صفرًا بلا سبب ويضلّل الطالب.

ج4. بإكمال أي درس (Unlock.lessonCompleted). قبل ذلك حالتها locked.

ج5. لأننا لا نقيس وقت الدراسة؛ نعدّ فقط ما نعرفه (محاولات وأجزاء منجزة).

مصطلحات

المصطلح	المعنى
ProgressState	وصف ما تعرضه «تقدّمي»، مشتق من القاعدة
Badge	شارة تُشتق من البيانات
WardrobeItem	عنصر في الخزانة (إكسسوار أو لون)
PurchaseResult	نتيجة محاولة الشراء
Equipped	مُلبَس الآن

ما بعد هذه المرحلة

رئيسية الصغار ولعبة العدّ في **المرحلة 9** (وتكسبان نجومًا تُصرف هنا). والمزامنة والتنزيل والمشاركة في الأدلة الإضافية.